Faculty of Engineering — Zagazig University
Computer and Systems Engineering Program

CSE 2026: IOT Laboratory

ESP8266 Lamp Control System

Name	**Section**	**Nummber**
Youssef Reda	2	20
Mohamed Moustafa	2	21
Osamn Mohamed	1	22

Academic Year 2026/2027

Contents

Introduction **3**
Project Overview . . . 3
Problem Statement . . . 3
Project Objectives . . . 3

System Requirements and Technologies **4**
Hardware Components . . . 4
Software and Development Tools . . . 4
Technologies and Libraries . . . 4

System Design and Architecture **6**
Overall System Architecture . . . 6
System Components and Responsibilities . . . 6
Communication Protocol and Message Format . . . 6
System Workflow and Data Flow . . . 6

System Implementation **14**
Backend Implementation . . . 14
Web Dashboard Implementation . . . 14
ESP8266 Firmware Implementation . . . 14
Lamp State and Brightness Control . . . 14
WebSocket Communication and Synchronization . . . 14

Testing and Results **14**

References **15**

Introduction

Project Overview

This project presents an IoT-based lamp control system using an ESP8266 microcontroller and a web-based dashboard. The system enables users to control the built-in LED remotely through a WebSocket connection, allowing real-time updates to the lamp state and brightness.

Problem Statement

Controlling an electronic device directly through physical interaction limits the flexibility of remote operation. This project addresses this limitation by providing a web interface that communicates with an ESP8266 over Wi-Fi through a Node.js backend, enabling remote lamp control within the connected network.

Project Objectives

The main objectives of this project are to:

- Establish Wi-Fi connectivity between the ESP8266 and the system.
- Develop a web dashboard for lamp control and brightness adjustment.
- Implement WebSocket communication for exchanging commands and state updates.
- Synchronize the displayed lamp state with the backend state.

System Requirements and Technologies

Hardware Components

- ESP8266 development board with a built-in LED.
- Computer for running the Node.js backend and accessing the web dashboard.
- Wi-Fi network for communication between the ESP8266 and the backend.

Software and Development Tools

- Node.js runtime environment.
- Web browser for accessing the control dashboard.
- Arduino IDE for developing and uploading the ESP8266 firmware.
- Code editor for developing the backend and frontend.

Technologies and Libraries

- **Express.js:** Serves the dashboard and handles HTTP routes.
- **ws:** Provides WebSocket communication on the Node.js server.
- **WebSockets:** Enables bidirectional communication between the dashboard, backend, and connected device.
- **ArduinoJson:** Parses JSON messages received by the ESP8266.
- **ESP8266WiFi:** Connects the microcontroller to the Wi-Fi network.
- **WebSocketsClient_Generic:** Provides WebSocket client functionality on the ESP8266.
- **HTML, CSS, and JavaScript:** Build the web dashboard and its interactive controls.

System Design and Architecture

Overall System Architecture

The system consists of a web dashboard, a Node.js backend, an ESP8266 microcontroller, and a built-in LED. The dashboard communicates with the backend through WebSockets. The backend maintains the lamp state and broadcasts state updates to all connected WebSocket clients, including the dashboard and ESP8266. The microcontroller processes the received state and controls the LED output accordingly.

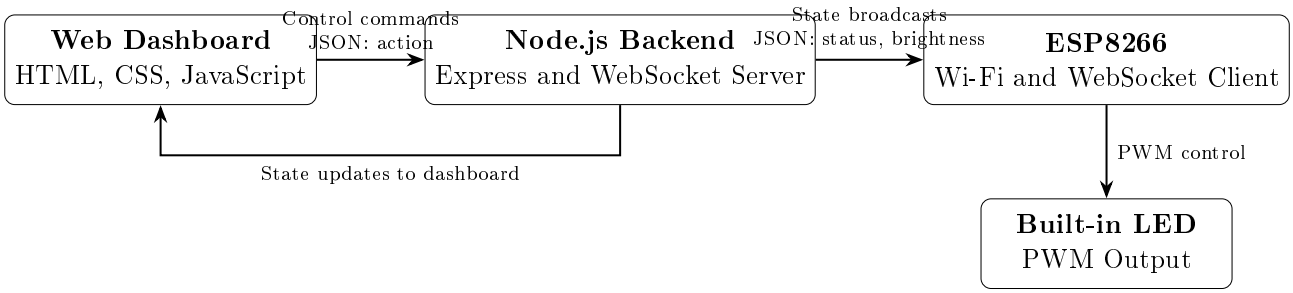


Figure 1: Overall architecture of the ESP8266 lamp control system.

System Components and Responsibilities

The web dashboard provides controls for switching the lamp on and off and adjusting its brightness. The Node.js backend processes incoming commands and maintains the lamp state in memory through the lamp service. The WebSocket server broadcasts the updated state to connected clients. The ESP8266 receives these updates and applies the corresponding output to its built-in LED.

Communication Protocol and Message Format

WebSocket communication is used to exchange messages between the dashboard and the backend and to deliver state updates to the connected ESP8266. Messages are encoded in JSON format.

The dashboard sends commands containing an `action` field, such as `on`, `off`, or `change_brightness`, with a brightness value when applicable. After processing a command, the backend broadcasts a state object containing the `status` and `brightness` fields. The ESP8266 uses these fields to determine the required LED output.

System Workflow and Data Flow

When a user interacts with the dashboard, the browser sends a command to the backend. The backend updates the in-memory lamp state and broadcasts the resulting state to all connected WebSocket clients. The dashboard updates its interface, while the ESP8266 processes the broadcast and adjusts the LED output.

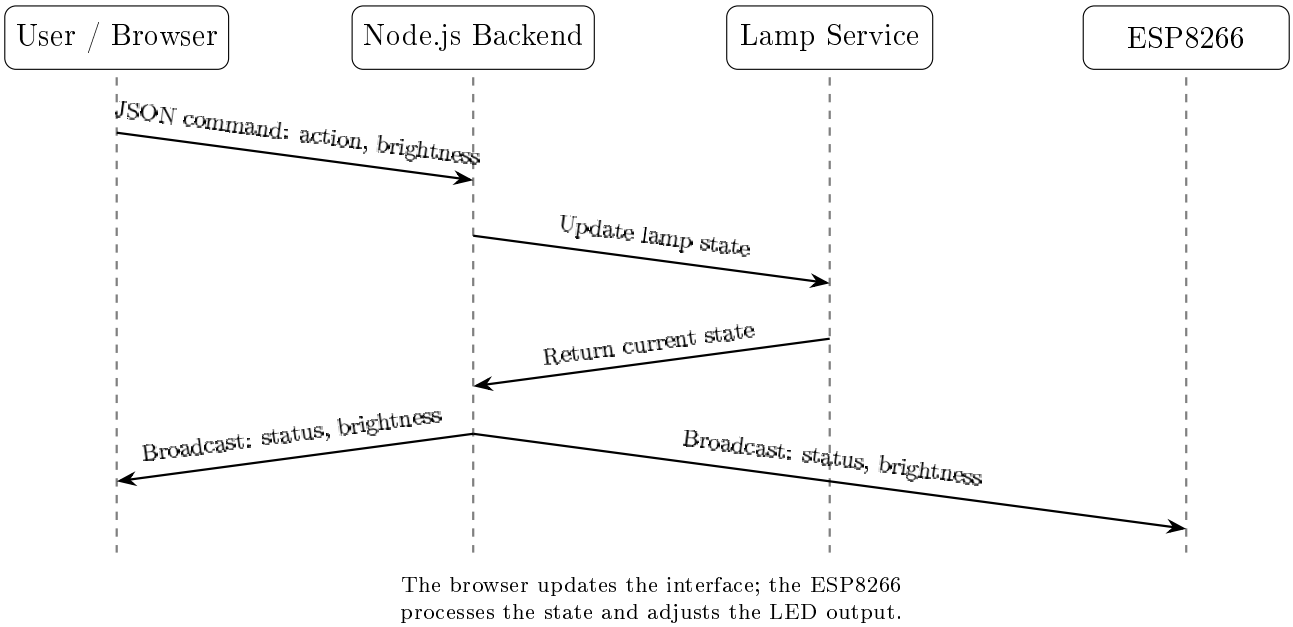


Figure 2: Sequence of a lamp control command and state broadcast.

System Implementation

Backend Implementation

The Node.js backend uses Express.js to serve the dashboard and handle HTTP requests. A WebSocket server runs on port 2000, while a dedicated lamp service maintains the current lamp status and brightness in memory.

Listing 1: HTTP and WebSocket Server Initialization

```
const app = require('./server/server.js');
const http = require('node:http');
const createWebSocketServer = require('./websocket/websocket.js');

const server = http.createServer(app);
createWebSocketServer(server);

const PORT = 2000;

server.listen(PORT, () => {
    console.log(`Server is running at port ${PORT}`);
});
```

Listing 2: Express.js Application and HTTP Route Configuration

```
const express = require('express');
const app = express();
const path = require('node:path')

app.use(express.static(path.join(__dirname, '..', 'views')));

app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, '..', 'views', 'index.html'));
});

// Testing endpoint

app.get('/api/test', (req, res) => {
    res.json({
        status: 'ok',
        message: "Backend is runnig"
    });
});

module.exports = app;
```

Web Dashboard Implementation

The dashboard is built with HTML, CSS, and JavaScript. It provides power and brightness controls, displays the connection status, and sends JSON commands to the backend through WebSockets.

Listing 3: Web Dashboard Controls and WebSocket Communication

```
const HOST = location.origin.replace(/^http/, 'ws');
const ws = new WebSocket(HOST);

.....

ws.onopen = function () {
    console.log("Connected to WebSocket server");
    setConnectionConnected();
    turnLampOn(255);
};

powerBtn.onclick = () => {
    if (isOn) {
        turnLampOff();
    } else {
        turnLampOn(255);
    }
};

slider.addEventListener('input', () => {
    brightnessVal.textContent = slider.value;
});

slider.addEventListener('change', () => {
    const brightness = Number(slider.value);

    if (brightness === 0) {
        turnLampOff();
    } else {
        setLampOnUI(brightness);

        lampCommand['action'] = "change_brightness";
        lampCommand['brightness'] = brightness;
        if (ws.readyState === WebSocket.OPEN) {
            ws.send(JSON.stringify(lampCommand));
        }
    }
});

ws.onmessage = function (event) {
    let lampStatus = JSON.parse(event.data);

    if (lampStatus.status === "on") {
        brightness = lampStatus.brightness;
        setLampOnUI(brightness);
    } else if (lampStatus.status === "off") {
        setLampOffUI();
    }
};

```

```
ws.onerror = function () {
    setConnectionDisconnected();
};

ws.onclose = function () {
    console.log("WebSocket connection closed");
    setConnectionDisconnected();
    setLampOffUI();
};
```

ESP8266 Firmware Implementation

The ESP8266 firmware uses the Wi-Fi, WebSocket client, and ArduinoJson libraries. It connects to the configured Wi-Fi network, receives JSON state messages, and controls the built-in LED. A five-second WebSocket reconnection interval is configured.

Listing 4: ESP8266 LED Control and PWM Implementation

```
#include <ESP8266WiFi.h>
#include <WebSocketsClient_Generic.h>
#include <ArduinoJson.h>

// Configurations
constexpr char WIFI_SSID[] = "SSID";
constexpr char WIFI_PASSWORD[] = "PASSWORD";

constexpr char SERVER[] = "SERVER_ADDRESS";
constexpr uint16_t SERVER_PORT = 2000;
constexpr char SERVER_PATH[] = "/";

constexpr uint8_t LED_PIN = LED_BUILTIN;

// WebSocket
WebSocketsClient webSocket;

// Command execution
void executeCommand(JsonDocument &json)
{
    const char *status = json["status"];
    int brightness = json["brightness"] | 0;

    if (status == nullptr)
    {
        Serial.println("Warning: Received JSON without a 'status' field.");
        return;
    }

    Serial.print("Executing status: ");
    Serial.print(status);
    Serial.print(" | brightness: ");
    Serial.println(brightness);

    if (strcmp(status, "on") == 0)
    {
        // LED_BUILTIN is usually active-low on ESP8266, so 255 - brightness works
```

```
    perfectly
        analogWrite(LED_PIN, 255 - brightness);
        Serial.println("LED updated (ON)");
    }
    else if (strcmp(status, "off") == 0)
    {
        analogWrite(LED_PIN, 255);
        Serial.println("LED turned OFF");
    }
}

// JSON message
void handleMessage(uint8_t *payload, size_t length)
{
    Serial.print("Received raw payload: ");
    for (size_t i = 0; i < length; i++)
    {
        Serial.print((char)payload[i]);
    }
    Serial.println();

    JsonDocument json;

    DeserializationError error =
        deserializeJson(json, payload, length);

    if (error)
    {
        Serial.print("Invalid JSON: ");
        Serial.println(error.c_str());
        return;
    }

    executeCommand(json);
}

// WebSocket events
void onWebSocketEvent(
    WStype_t type,
    uint8_t *payload,
    size_t length)
{
    switch (type)
    {
    case WStype_CONNECTED:
        Serial.print("[WSc] Connected to url: ");
        Serial.println((char *)payload);
        break;

    case WStype_DISCONNECTED:
        Serial.println("[WSc] Disconnected from server.");
        break;

    case WStype_TEXT:
        Serial.println("[WSc] Received TEXT message.");
        handleMessage(payload, length);
        break;

    case WStype_ERROR:
        Serial.println("[WSc] Error occurred!");
```

```
97          break;
98
99      default:
100         Serial.print("[WSc] Unhandled event type: ");
101         Serial.println(type);
102         break;
103     }
104 }
105
106 void setup()
107 {
108     Serial.begin(921600);
109
110     pinMode(LED_PIN, OUTPUT);
111
112     // LED off
113     analogWrite(LED_PIN, 255);
114
115     // Wi-Fi
116     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
117
118     Serial.print("Connecting to Wi-Fi");
119
120     while (WiFi.status() != WL_CONNECTED)
121     {
122         delay(250);
123         Serial.print(".");
124     }
125
126     Serial.println();
127
128     Serial.print("Wi-Fi connected. IP: ");
129     Serial.println(WiFi.localIP());
130
131     // WebSocket
132     webSocket.begin(
133         SERVER,
134         SERVER_PORT,
135         SERVER_PATH);
136
137     webSocket.onEvent(onWebSocketEvent);
138     webSocket.setReconnectInterval(5000);
139
140     Serial.println("WebSocket client started.");
141 }
142
143 void loop()
144 {
145     webSocket.loop();
146 }
```

Lamp State and Brightness Control

The backend stores the lamp status and brightness, using values from 0 to 255. Turning the lamp on sets the brightness to 255, while turning it off sets it to zero. The ESP8266 uses `analogWrite()` and an active-low PWM mapping to control the built-in LED.

Listing 5: Lamp State Management and Control Functions

```
const lamp = {
    status: "off",
    brightness: 0
};

function turnOn() {
    lamp.status = "on";
    lamp.brightness = 255;
};

function turnOff() {
    lamp.status = "off";
    lamp.brightness = 0;
};

function setBrightness(value) {
    lamp.status = "on";
    lamp.brightness = value;
};

function getStatus() {
    return lamp;
};

module.exports = {
    turnOn,
    turnOff,
    setBrightness,
    getStatus
};
```

WebSocket Communication and Synchronization

The dashboard sends commands using the `action` field. The backend updates the lamp state and broadcasts `status` and `brightness` to connected clients. The dashboard updates its interface, and the ESP8266 adjusts the LED output based on the received state. The backend does not receive explicit confirmation of the physical LED state.

Listing 6: WebSocket Message Handling and State Broadcasting

```
const lampService = require('../services/lampService.js');
const SocketServer = require('ws').Server;

function createWebSocketServer(server) {
    const wss = new SocketServer({ server });

    wss.on("connection", (ws) => {
        console.log('Client connected');

        ws.on('message', (msg) => {
            const lampStatus = msg.toString();
            const lampCommand = JSON.parse(lampStatus);

            if (lampCommand.action === "on") {
                lampService.turnOn();
            } else if (lampCommand.action === "off") {
                lampService.turnOff();
            } else if (lampCommand.action === "change_brightness") {
                lampService.setBrightness(lampCommand.brightness);
            }

            const lampState = lampService.getStatus();

            broadcast(wss, lampState);

            console.log("Message form client: ", lampStatus);
        });

        ws.on('close', () => {
            console.log('Client disconnected');
        });
    });

    return wss;
};

function broadcast(wss, lampState) {
    const message = JSON.stringify(lampState);

    wss.clients.forEach(function (client) {
        if (client.readyState === client.OPEN) {
            client.send(message);
        } else {
            console.log("Clinet off");
        }
    });
}

module.exports = createWebSocketServer;
```

Testing and Results

The system was evaluated by checking the interaction between the web dashboard, the Node.js backend, and the ESP8266 controller. The main operations considered were establishing the WebSocket connection, switching the lamp on and off, adjusting brightness, and updating the displayed lamp state. Screenshots of the implemented system and its observed behavior are provided below to document the testing process.

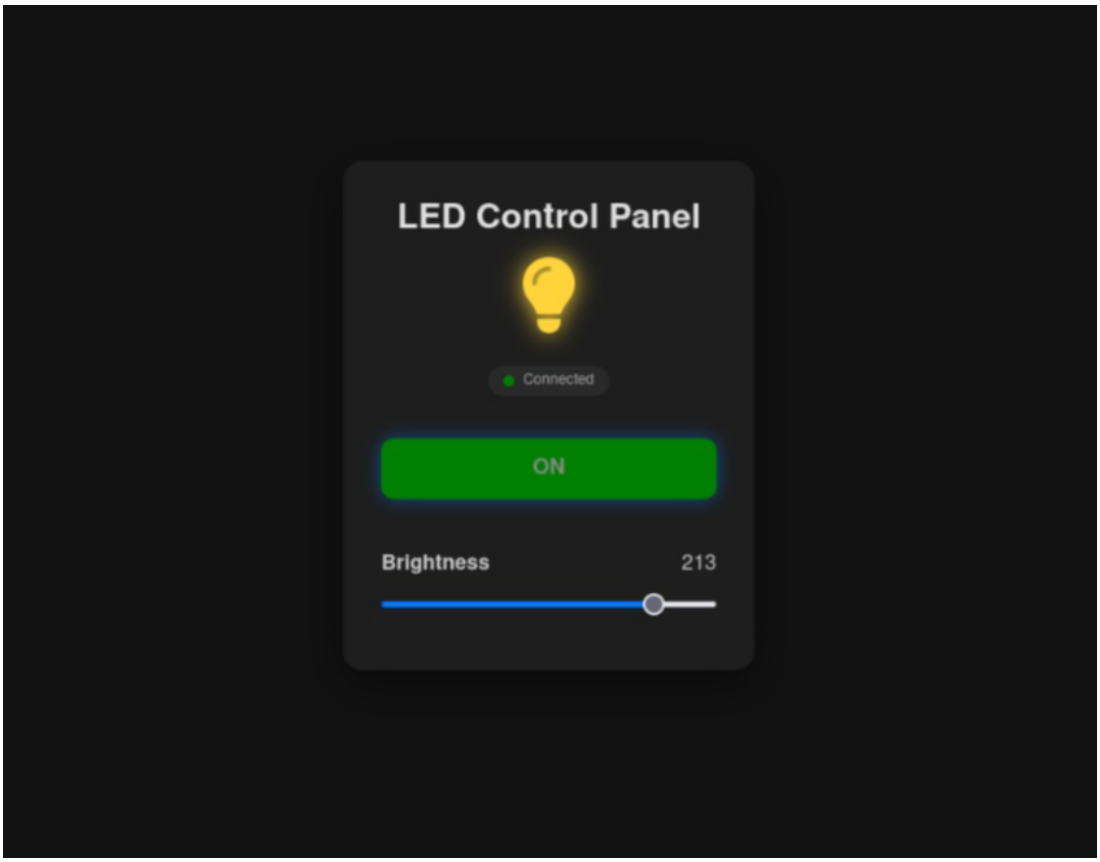


Figure 3: Web Dashboard Interface



Figure 4: WebSockets connection

References

[1] Youssef.Reda, *CSE3-IOT-Labs: IoT Laboratory Projects*. GitHub Repository. Available at: https://github.com/Youssef-Reda440/CSE3-IOT-Labs

[2] IoT Laboratory Teaching Assistant, *IoT Laboratory Tutorials*. YouTube Playlist. Available at: https://youtube.com/playlist?list=PLKrTtHfjR9toHOJ1ZnHn-700OwPN-eG_N